```cpp
// 1. Write a menu driven program with 4 options (Insert, Delete, Display, and Exit) to demonstrate the workin
g of Queues using linked-list.

#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int d){ data=d; next=nullptr; }
};

struct QueueLL {
    Node* front;
    Node* rear;
    QueueLL() { front = rear = nullptr; }

    void insert(int val) {
        Node* newnode = new Node(val);
        if (rear == nullptr) {
            front = rear = newnode;
        } else {
            rear->next = newnode;
            rear = newnode;
        }
    }
    void deleteq() {
        if(front == nullptr) {
            cout << "Queue Underflow\n";
            return;
        }
        cout << "Deleted: " << front->data << "\n";
        Node* temp = front;
```

```
        if(front == nullptr) rear = nullptr;
        delete temp;
    }
    void display() {
        if(front == nullptr) cout << "Queue Empty\n";
        else {
            Node* curr = front;
            while(curr) {
                cout << curr->data << " ";
                curr = curr->next;
            }
            cout << "\n";
        }
    }
};

int main() {
    QueueLL q;
    int ch, val;
    while(1) {
        cout << "1.Insert 2.Delete 3.Display 4.Exit\nEnter choice: ";
        cin >> ch;
        if(ch==1) {
            cout << "Enter value: "; cin >> val;
            q.insert(val);
        } else if(ch==2) q.deleteq();
        else if(ch==3) q.display();
        else if(ch==4) break;
        else cout << "Invalid!\n";
    }
    return 0;
}
```

```
~/Documents/dslab  master !1 ?8  g++ assignment6_q1.cpp
~/Documents/dslab  master !1 ?8  ./a.out
1.Insert 2.Delete 3.Display 4.Exit
Enter choice: 1
10
1
20
1
30
3
2
3
2
2
3
4
Enter value: 1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Enter value: 1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Enter value: 1.Insert 2.Delete 3.Display 4.Exit
Enter choice: 10 20 30
1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Deleted: 10
1.Insert 2.Delete 3.Display 4.Exit
Enter choice: 20 30
1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Deleted: 20
1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Deleted: 30
1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Queue Empty
1.Insert 2.Delete 3.Display 4.Exit
Enter choice:
```

```
// 2. Write a menu driven program with 4 options (Insert, Delete, Display, and Exit) to demonstrate the work
g of Circular Queues (arrays).

#include <bits/stdc++.h>
using namespace std;
#define MAX 100

struct CircularQueue {
    int arr[MAX];
    int front, rear, size;
    CircularQueue(int sz=5){ size=sz; front=rear=-1; }

    void insert(int val) {
        if((front==0 && rear==size-1) || (rear==(front-1)%(size-1))) {
            cout << "Queue Overflow\n";
            return;
        }
        if(front==-1) {
            front=rear=0;
        } else if(rear==size-1 && front!=0) {
            rear=0;
        } else {
            rear++;
        }
        arr[rear]=val;
    }
    void deleteq() {
        if(front==-1) {
            cout << "Queue Underflow\n";
            return;
        }
        cout << "Deleted: " << arr[front] << "\n";
```

```
        } else if(front==size-1) {
            front=0;
        } else {
            front++;
        }
    }
    void display() {
        if(front==-1) cout << "Queue Empty\n";
        else {
            int i=front;
            cout << "Queue: ";
            while(1) {
                cout << arr[i] << " ";
                if(i==rear) break;
                i=(i+1)%size;
            }
            cout << "\n";
        }
    }
};

int main() {
    int qsize;
    cout << "Enter circular queue size (up to 100): ";
    cin >> qsize;
    CircularQueue q(qsize);
    int ch, val;
    while(1) {
        cout << "1.Insert 2.Delete 3.Display 4.Exit\nEnter choice: ";
        cin >> ch;
        if(ch==1) {
            cout << "Enter value: "; cin >> val;
```

```
        else {
            int i=front;
            cout << "Queue: ";
            while(1) {
                cout << arr[i] << " ";
                if(i==rear) break;
                i=(i+1)%size;
            }
            cout << "\n";
        }
    }
};

int main() {
    int qsize;
    cout << "Enter circular queue size (up to 100): ";
    cin >> qsize;
    CircularQueue q(qsize);
    int ch, val;
    while(1) {
        cout << "1.Insert 2.Delete 3.Display 4.Exit\nEnter choice: ";
        cin >> ch;
        if(ch==1) {
            cout << "Enter value: "; cin >> val;
            q.insert(val);
        } else if(ch==2) q.deleteq();
        else if(ch==3) q.display();
        else if(ch==4) break;
        else cout << "Invalid!\n";
    }
    return 0;
}
```

```
Enter circular queue size (up to 100): 5
1
11
1
22
1
33
1
44
1
55
3
2
3
1
66
2
2
2
3
4
1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Enter value: 1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Enter value: 1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Enter value: 1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Enter value: 1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Enter value: 1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Queue: 11 22 33 44 55
1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Deleted: 11
1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Queue: 22 33 44 55
1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Enter value: 1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Deleted: 22
1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Deleted: 33
1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Deleted: 44
1.Insert 2.Delete 3.Display 4.Exit
Enter choice: Queue: 55 66
1.Insert 2.Delete 3.Display 4.Exit
Enter choice:
```